

Maya_Ros_Sim (Maro) 생태계 최종 설계도

결함 허용(Fault-Tolerant) 및 자가 학습(Self-Healing) 기반 4-Tier + 1 마이크로서비스 아키텍처

💡 **핵심 철학:** 단일 프로세스의 한계 극복, 극한의 성능 최적화, 파편화된 로깅, 순환 검증을 통한 무결성 확보, 그리고 메모이제이션(Memoization) 기반의 자가 학습 디버깅.

1. 시스템 전체 구성 (The 5 Pillars)

본 아키텍처는 책임을 철저히 분리한 5개의 독립된 모듈로 유기적으로 작동합니다.

1. onfix_Maro (내부 관측 및 보강)

- **환경:** Maya In-Process (C++)
- **역할:** 디렉터리 경로 초기 인덱싱(캐싱), 실시간 하드웨어/메모리 에러 관측.
- **크래시 동작:** 분석 금지. 가장 가벼운 **파편 1(Seed)** 생성.
- **재부팅 동작:** **ghost** 가 모아온 초벌 데이터를 받아 C++ 코어 정보를 **보강(Enrich)**.

2. offix_Maro (사후 분석 및 중계)

- **환경:** 백그라운드 Out-of-Process (Python)
- **역할:** 지연 평가(Lazy Evaluation) 기반 디렉터리/코드 파싱, 종속성 추적.
- **크래시 동작:** **파편 2(마지막 접근 파일)** 디스크 기록.
- **재부팅 동작:** **book_Maro** 에 정답을 질의하여 연산 절약. 정답이 없으면 직접 분석.

3. ghost_Maro (예측 및 어셈블러)

- **환경:** OS Daemon (C++) / E-Core 샌드박스
- **역할:** 지속적 새도우 테스트. 치명적 오류 **예측 시 글로벌 비상 시그널 격발**.
- **재부팅 동작:** 파편화된 데이터 조각을 수집(Assemble). 순환 검증 사이클의 시작과 끝을 담당.

4. boad_Maro & 5. book_Maro (시각화 및 자가학습)

- **환경:** Electron + React 데스크톱 오버레이 UI
- **boad_Maro:** 사용자 UI 대시보드. 크래시 직전 **파편 3(유저 액션)** 기록. OSbridge 역할.
- **book_Maro (지식 DB):** 과거 분석된 에러 패턴(Hash) 과 해결책을 캐싱. 중복 에러 발생 시 **offix** 에 **0.1초** 만에 정답 제공 (연산 100% 절약).

2. 극한의 최적화: 크래시 방어 및 분산 로깅 메커니즘

[문제 정의] 시스템이 셧다운 되기 직전(수 밀리초 전)에 무거운 분석 로직을 돌리거나 하나의 거대한 JSON 파일을 쓰려고 하면 CPU/I/O 병목으로 데이터가 유실(Corrupted)됨.

[해결: 3분할 병렬 분산 로깅]

1. **ghost** 가 새도우 테스트 중 커널 패닉을 예측하고 **글로벌 비상 이벤트(Event Trigger)**를 격발.

2. 통신(IPC)조차 사치. `onfix`, `offix`, `boad` 는 즉각 하던 일을 멈추고 각자 지정된 디스크 구역에 자신만 아는 **최소한의 파편(Fragment) 데이터**를 강제 플러시(FILE_FLAG_WRITE_THROUGH).
3. 각자의 파편 기록 완료 후 시스템 셧다운. (**병목 현상 제로, 저장 속도 3배 향상**)

3. 부활 시나리오: 순환 검증 조립 (Circular Validation)

PC가 재부팅되고 자원이 넉넉해졌을 때, 흩어진 단서를 조립하여 완벽한 컨텍스트를 만드는 '릴레이 검증' 과정입니다.

```
[T0] ghost_Maro : 3개의 파편 수거 → [초벌 조립본 생성]
      ↓ (Send to onfix)
[T1] onfix_Maro : 초벌본 수신 → 하드웨어 레지스터/메모리 단서 [보강]
      ↓ (Send to offix)
[T2] offix_Maro : 보강본 수신 → (book_Maro에 정답 질의) → 연관 문서/경로 분석 [최종 완성]
      ↓ (Send back to ghost)
[T3] ghost_Maro : 완성본 수신 → boad_Maro로 송출
```

4. 경험 기반 자가 치유 시스템 (book_Maro)

단순 분석 유틸리티를 넘어, 과거의 경험을 바탕으로 시스템 자원을 극도로 아끼는 메모이제이션(Memoization) 디버깅 개념입니다.

- **작동 원리:** boad가 에러를 렌더링할 때, book_Maro (로컬 DB)에 [에러 해시 키 : 분석 결과] 쌍으로 학습시킵니다.
- **성능 혁신:** 이후 동일한 에러가 발생하면, offix는 무거운 디스크 스캔을 하지 않고 book_Maro에 질의하여 즉시 정답을 받아옵니다.
- **결과:** 반복되는 에러에 대한 디버깅 연산 비용 0%, 응답 속도 빛의 속도.

5. 하드웨어 자원 격리 (Sandbox)

메인 3D 프로그램(Maya)의 성능(프레임레이트)을 절대 깎아먹지 않기 위한 로우레벨 제어입니다.

- **스레드 선호도 (Thread Affinity):** ghost_Maro의 백그라운드 예측 연산은 P-Core(고성능 코어)를 배제하고 E-Core(고효율 코어)에만 할당됩니다.
- **메모리 제한 (Memory Cap):** Windows Job Object API를 통해 ghost의 새도우 테스트 메모리 상한을 256MB 등으로 제한, 가상 테스트 중 메모리가 터지더라도 메인 시스템은 보호됩니다.